

Лабораторная 5

ООП, магические методы.

Дан класс Vector.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

Для продвинутого уровня реализуйте задания для вектора произвольной длины, т.е. вместо двух координат x и y в классе **Vector** будет список координат.

1. Реализуйте метод строкового представления **__str__**: он должен возвращать строку в формате "Vector(x, y)". Например: "Vector(3, 4)".

2. **Сравнение векторов.** Реализуйте метод проверки на равенство **__eq__**

Два вектора равны, если совпадают их координаты x и y .

Проверка: `Vector(1, 2) == Vector(1, 2)` должно возвращать `True`, а `Vector(1, 2) == Vector(2, 1)` — `False`.

При сравнении с объектом другого типа (например, числом или строкой) метод должен возвращать `False`, а не выбрасывать ошибку.

3. Реализуйте **возможность складывать и вычитать** векторы **__add__**, **__sub__**.

$v1 + v2 \rightarrow \text{Vector}(v1.x + v2.x, v1.y + v2.y)$

$v1 - v2 \rightarrow \text{Vector}(v1.x - v2.x, v1.y - v2.y)$

Методы должны возвращать новый объект Vector.

4. Реализуйте умножение вектора на число **__mul__**, **__rmul__**. Например,:

__mul__: Позволяет писать `vector * 5`.

__rmul__: Позволяет писать `5 * vector` (обратное умножение, когда слева число, а справа объект).

Каждая координата умножается на число. `Vector(1, 2) * 3  $\rightarrow$  Vector(3, 6)`.

Если пытаются умножить на нечисловой тип, выбросьте `TypeError`.

5. Реализуйте метод для вычисления евклидовой длины вектора **__abs__**.

При вызове `abs(vector)` должно возвращаться число (float), равное $\sqrt{x^2 + y^2}$.

6. Реализуйте унарный минус **__neg__**.

Выражение `-vector` должно возвращать новый вектор с противоположными координатами.

Пример: `-Vector(3, -4)  $\rightarrow$  Vector(-3, 4)`.

7. Реализуйте метод **__len__** для вычисления размерности вектора.

8. Реализуйте метод **__bool__**. Вектор считается «ложным» (`False`), если он нулевой (обе координаты равны 0). В противном случае — `True`.